

Note de livraison - Projet JEE

"BanqueWeb"



Yassine HAMROUNI

1. Pré-requis pour l'hébergement en production

Pour déployer et faire fonctionner l'application BanqueWeb dans un environnement de production chez le client, l'infrastructure suivante est requise :

- **Système d'Exploitation** : Serveur Linux (ex: Ubuntu Server, CentOS) ou Windows Server.
- **Environnement Java (JRE/JDK)** : Java 8 ou version supérieure (Java 11 LTS recommandé). L'application a été développée pour être compatible avec les standards J2EE 7/8.
- **Serveur d'Application Web : Apache Tomcat 9.0.x.**
 - *Note importante* : L'application utilise le package `javax.servlet`. Elle n'est pas compatible nativement avec Tomcat 10+ (qui utilise `jakarta.servlet`) sans migration de code. Il est impératif d'utiliser une version 9.
- **Base de Données : MySQL Server 8.0** (ou MariaDB équivalent).
 - Le schéma de base de données (BANQUE) doit être initialisé avec les tables COMPTE, OPERATIONS et UTILISATEURS.
 - Un utilisateur dédié avec les droits SELECT, INSERT, UPDATE sur cette base doit être créé.
- **Bibliothèques tierces (Dépendances)** :
 - Connector JDBC : `mysql-connector-j` (compatible MySQL 8).
 - JSTL : `jstl-1.2.jar` (Standard Tag Library) pour l'affichage JSP.
- **Configuration Serveur** :
 - Configuration d'une ressource JNDI (`jdbc/Banque`) dans le `context.xml` du serveur pour activer le Pool de Connexions, garantissant la performance

2. État d'avancement du projet

Je suis parvenu à valider l'**Exercice 17** du sujet.

Fonctionnalités implémentées :

- Architecture MVC complète (JavaBeans, Servlets, JSP).
- Connexion à la base de données via un **Pool de Connexions** (optimisation des performances).
- Recherche et Consultation d'un compte bancaire.
- Opérations de Crédit et Débit avec gestion transactionnelle (Commit/Rollback).
- Historique des opérations filtré par dates.
- Affichage dynamique via JSTL (abandon des scriptlets Java dans le HTML).
- Gestion centralisée des erreurs (Pages 404, 500 et Exceptions Métier personnalisées).
- Sécurisation de l'application via un Filtre d'authentification (interception des accès non autorisés).
- Mise en place de la couche technique pour la gestion des Rôles et Utilisateurs (Table UTILISATEURS et Bean BUtilisateurs).

3. Libertés d'implémentation et Choix techniques

Pour répondre au cahier des charges tout en assurant la maintenabilité et la robustesse de l'application, j'ai pris les libertés suivantes :

- **Sécurité via Filtre (Filter API)** : Plutôt que de vérifier la session utilisateur dans chaque méthode de chaque Servlet (ce qui crée de la redondance), j'ai implémenté un Filtre (FAuthentication). Cela permet de sécuriser globalement l'accès aux pages JSP et aux Servlets, rendant l'application plus sûre par "design".
- **Gestion des Exceptions** : J'ai créé une architecture d'exceptions personnalisée (TraitementException et MessagesDErrors). Cela permet de séparer clairement l'erreur technique (loguée pour l'admin) du message fonctionnel (affiché à l'utilisateur), améliorant l'expérience utilisateur.
- **Utilisation de la JSTL** : J'ai anticipé les bonnes pratiques en remplaçant systématiquement le code Java intrusif dans les vues (<% . . . %>) par des balises JSTL (<c:forEach>, <c:if>), ce qui respecte mieux le pattern MVC.

4. Commentaires et Retour d'expérience

Réaliser ce projet seul a été un vrai défi. J'ai dû tout gérer, de la configuration d'Eclipse et Tomcat jusqu'au code final, et honnêtement, les problèmes techniques m'ont pris beaucoup de temps.

C'est pourquoi je me suis arrêté à l'étape 17. J'ai préféré rendre un projet propre, stable et bien codé plutôt que d'essayer de tout finir à la va-vite. J'ai vraiment fait le maximum pour produire un résultat de qualité malgré les circonstances, et j'espère que le fait que je sois seul sera pris en compte dans la note. Merci de votre compréhension.